HomeServe Technologies — Architecture & Implementation Plan

1. Executive Summary & Requirements Analysis

HomeServe Technologies is an enterprise-grade, multi-tenant SaaS marketplace connecting homeowners and businesses (Customers) with verified Service Providers and field Technicians, governed by a Super Admin.

Core Stakeholders & Role Matrix

1. **Super Admin**: Full platform governance, provider KYC verification, category/service catalog management, financial oversight (commissions, payouts, platform revenue), coupon management, customer complaints resolution, and system analytics.
2. **Customer**: On-demand search, filtering, verified provider comparison, real-time slot availability, instant booking with atomic slot reservation, online payment (Razorpay / test gateway), live booking status tracking, invoices, post-service ratings/reviews, and complaint management.
3. **Service Provider**: Business profile management, KYC document uploads, service offering catalog with dynamic pricing, service areas and working hours/availability management, incoming booking requests (Accept / Reject), technician job dispatching, work evidence (before/after photos), earnings and commission statements.
4. **Technician**: Field mobility dashboard, assigned job queue, customer contact details and navigation address, job status updates (*Accepted*, *On The Way*, *Arrived*, *In Progress*, *Completed*), work remarks, proof-of-work completion uploads.

2. Technology Stack & Key Libraries

Layer	Technology	Details
Frontend Framework	React 18+ with Vite	Fast HMR, optimized production build
Styling & UI	Bootstrap 5, Bootstrap Icons, Custom CSS	Modern SaaS aesthetic, responsive cards, modals, clean dark/light headers, badge system
Routing	React Router DOM v6	Nested layouts, role-based route guards (`CustomerRoute`, `ProviderRoute`, `TechnicianRoute`, `AdminRoute`)
State & HTTP	React Context API + Axios	Global `AuthContext`, `ToastContext`, Axios interceptors for JWT token injection and automatic 401 handling
Data Visualization	Recharts	Interactive revenue charts, booking volume graphs, category distributions
Backend Runtime	Node.js & Express.js	Modular REST architecture, MVC design pattern
Database	MongoDB & Mongoose ODM	Schema validation, composite indexes, atomic reservation transactions
Security & Auth	JWT (Access + Refresh tokens), bcryptjs, Helmet, Express Rate Limit, Express Validator, CORS	Strong password hashing, XSS/injection protection, rate limiting, granular RBAC
File Storage	Multer + Cloudinary (with local storage fallback)	Handles KYC docs, service banners, booking problem photos, and technician completion photos
Payments	Razorpay SDK (Order creation, signature verification, webhook support, fallback sandbox simulator)	Secure server-side signature verification (HMAC SHA-256)
Email/Notifications	Nodemailer (with Ethereal / console test transport fallback)	Automated booking confirmations, status updates, invoice emails

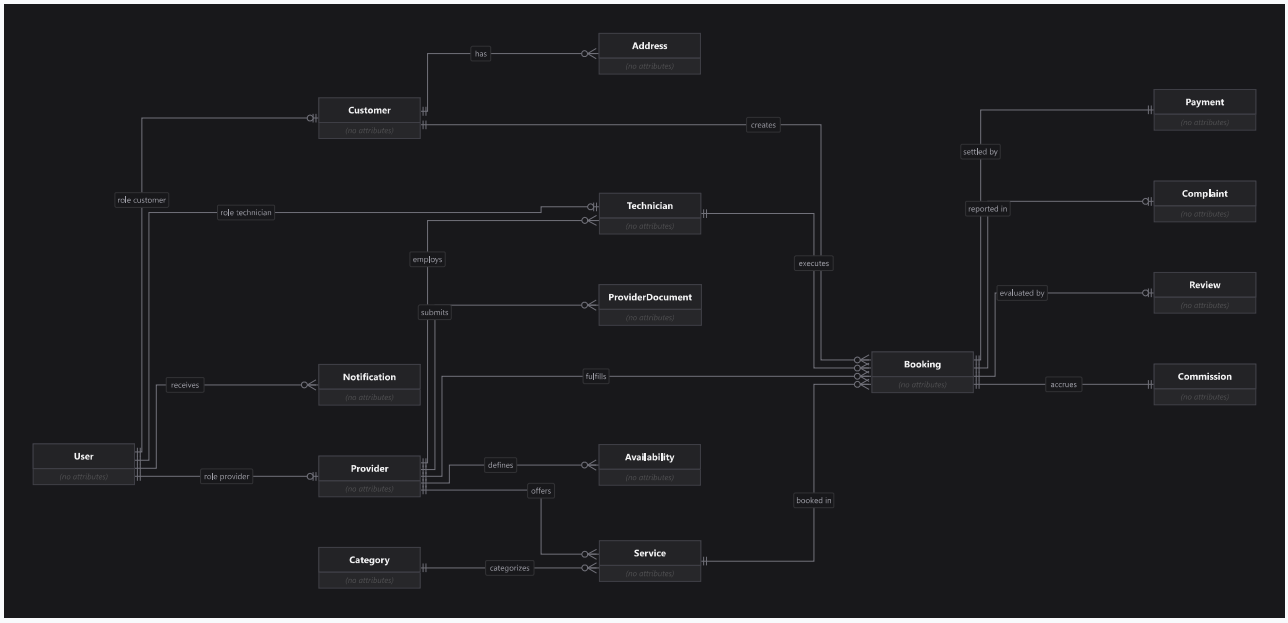
3. System Architecture & Directory Structure

```
d:\Home_Services_Booking\
├── backend\
│   ├── src\
│   │   ├── config\          # Database connection, Razorpay, Cloudinary, Nodemailer,
constants
│   │   ├── controllers\     # Admin, Auth, Booking, Category, Customer, Payment, Provider,
Review, Service, Technician
│   │   ├── middleware\      # authMiddleware (JWT & RBAC), errorHandler, rateLimiter,
uploadMiddleware, validationMiddleware
│   │   ├── models\          # User, Customer, Provider, Technician, Category, Service,
Booking, Payment, Transaction, Commission, Review, Complaint, Coupon, Availability,
ProviderDocument, Notification
│   │   ├── routes\          # adminRoutes, authRoutes, bookingRoutes, categoryRoutes,
customerRoutes, paymentRoutes, providerRoutes, reviewRoutes, serviceRoutes,
technicianRoutes
│   │   ├── services\        # bookingService, paymentService, emailService,
invoiceService, notificationService
│   │   ├── utils\           # apiResponse, AppError, seedData, logger, slotHelper
│   │   ├── validators\      # authValidators, bookingValidators, serviceValidators
│   │   └── server.js        # Express app initialization, middleware chains, router
mounts, DB connect
│   ├── .env.example
│   └── package.json
│
├── frontend\
│   ├── public\
│   ├── src\
│   │   ├── assets\          # Images, logos, icons, illustrations
│   │   ├── components\
│   │   │   ├── common\      # Navbar, Footer, Sidebar, StatCard, DataTable,
LoadingSpinner, EmptyState, ConfirmModal, AlertBadge, StarRating
│   │   │   ├── customer\    # BookingCard, ServiceSearchFilter, SlotPicker, AddressForm,
PaymentModal, ReviewModal
│   │   │   ├── provider\    # ServiceEditorModal, AssignTechModal, EarningsChart,
DocumentUploadCard
│   │   │   ├── technician\  # JobCard, StatusUpdaterModal, ProofUploadModal
│   │   │   └── admin\       # ProviderVerificationModal, CategoryModal, CommissionTable,
ComplaintModal
│   │   ├── context\         # AuthContext, ToastContext
│   │   ├── hooks\           # useAuth, useToast, useFetch
│   │   ├── layouts\         # PublicLayout, CustomerLayout, ProviderLayout,
TechnicianLayout, AdminLayout
│   │   ├── pages\
```

```
│   │   │   ├── public\      # Home, ServicesCatalog, ServiceDetails, ProviderProfile,
About, Contact, Login, Register, Unauthorized
│   │   │   ├── customer\    # CustomerDashboard, MyBookings, BookingDetails, Addresses,
Profile, CustomerComplaints
│   │   │   ├── provider\    # ProviderDashboard, ProviderServices, ProviderBookings,
ProviderTechnicians, Availability, ProviderEarnings, ProviderKYC
│   │   │   ├── technician\  # TechnicianDashboard, MyJobs, JobDetails, TechnicianProfile
│   │   │   └── admin\       # AdminDashboard, AdminProviders, AdminCustomers,
AdminTechnicians, AdminCategories, AdminServices, AdminBookings, AdminPayments,
AdminCommissions, AdminComplaints, AdminCoupons, AdminSettings
│   │   ├── routes\          # AppRoutes, ProtectedRoute
│   │   ├── services\        # api.js, authService.js, bookingService.js,
catalogService.js, paymentService.js, adminService.js, providerService.js,
technicianService.js
│   │   ├── utils\           # formatters.js (currency, dates), constants.js
│   │   ├── App.jsx
│   │   ├── main.jsx
│   │   └── index.css        # Custom theme variables, animations, glassmorphism,
responsive utilities
│   ├── index.html
│   ├── vite.config.js
│   └── package.json
└── README.md
```

4. Comprehensive Database Design (Mongoose Models)

Entity Relationship Architecture



Models & Schema Specifications

1. **User**: `_id`, `email`, `password` (bcrypt hash), `phone`, `role` (`['admin', 'customer', 'provider', 'technician']`), `firstName`, `lastName`, `avatar`, `isActive`, `isEmailVerified`, `refreshToken`, `createdAt`, `updatedAt`.
2. **Customer**: `_id`, `user` (ref: User), `savedAddresses` (array of subdocs or ref: Address), `emergencyContact`, `totalBookings`, `createdAt`.
3. **Provider**: `_id`, `user` (ref: User), `businessName`, `description`, `categoryIds` (refs: Category), `verificationStatus` (`['pending', 'approved', 'rejected']`), `rejectionReason`, `serviceAreas` (cities, pin codes), `commissionRate` (default 15%), `rating` (avg), `totalReviews`, `completedJobsCount`, `bankDetails` (accountNo, ifsc, accountHolder), `documentsVerified`, `createdAt`.
4. **Technician**: `_id`, `user` (ref: User), `provider` (ref: Provider), `specializations` (string array), `experienceYears`, `isAvailable`, `currentLocation` (lat, lng, addressString), `rating`, `completedJobsCount`, `createdAt`.
5. **Category**: `_id`, `name`, `slug`, `icon`, `image`, `description`, `isActive`, `sortOrder`, `createdAt`.
6. **Service**: `_id`, `provider` (ref: Provider), `category` (ref: Category), `title`, `slug`, `description`, `price`, `discountPrice`, `durationMinutes`, `coverImage`, `galleryImages`, `includedFeatures` (string array), `excludedFeatures` (string array), `isActive`, `rating`, `totalBookings`, `createdAt`.

7. **Address**: `_id`, `user` (ref: User), `type` (`['home', 'office', 'other']`), `streetAddress`, `landmark`, `city`, `state`, `pincode`, `contactPhone`, `isDefault`, `createdAt`.
8. **Booking**:
    - `_id`, `bookingReference` (unique indexed string, e.g., `HSB-20261104-XXXX`),
    - `customer` (ref: User), `provider` (ref: Provider), `technician` (ref: Technician, optional until assigned), `service` (ref: Service),
    - `bookingDate` (YYYY-MM-DD), `timeSlot` (e.g. "10:00 - 12:00"),
    - `serviceAddress` (embedded snapshot),
    - `problemDescription`, `problemImages` (array of URLs),
    - `status` (`['pending', 'accepted', 'rejected', 'technician_assigned', 'on_the_way', 'arrived', 'in_progress', 'completed', 'cancelled']`),
    - `statusHistory`: `[{ status, timestamp, updatedBy, remarks }]`,
    - `pricing`: `{ basePrice, discount, couponCode, platformFee, tax, totalAmount }`,
    - `paymentStatus` (`['pending', 'paid', 'failed', 'refunded']`),
    - `technicianRemarks`, `completionImages` (array of URLs), `completedAt`, `cancelledAt`, `cancellationReason`, `cancelledBy`.
9. **Payment**: `_id`, `booking` (ref: Booking), `user` (ref: User), `amount`, `currency` (INR), `paymentMethod` (`['razorpay', 'cash_on_delivery', 'card', 'upi']`), `razorpayOrderId`, `razorpayPaymentId`, `razorpaySignature`, `status` (`['created', 'authorized', 'captured', 'refunded', 'failed']`), `rawResponse`, `createdAt`.
10. **Transaction**: `_id`, `payment` (ref: Payment), `booking` (ref: Booking), `type` (`['payment', 'refund', 'payout', 'commission']`), `amount`, `status`, `referenceId`, `createdAt`.
11. **Commission**: `_id`, `booking` (ref: Booking), `provider` (ref: Provider), `totalBookingAmount`, `commissionPercentage`, `platformCommissionAmount`, `providerPayoutAmount`, `payoutStatus` (`['pending', 'settled']`), `settledAt`, `createdAt`.
12. **Review**: `_id`, `booking` (ref: Booking, unique), `customer` (ref: User), `provider` (ref: Provider), `service` (ref: Service), `rating` (1 to 5), `comment`, `replyFromProvider`, `isPublished`, `createdAt`.
13. **Complaint**: `_id`, `complaintTicket` (`CMP-XXXX`), `booking` (ref: Booking), `raisedBy` (ref: User), `provider` (ref: Provider), `subject`, `description`, `priority` (`['low', 'medium', 'high']`), `status` (`['open', 'under_investigation', 'resolved', 'closed']`), `adminResolutionRemarks`, `resolvedAt`, `createdAt`.
14. **Coupon**: `_id`, `code` (uppercase, unique), `discountType` (`['percentage', 'flat']`), `discountValue`, `minOrderAmount`, `maxDiscountAmount`, `validFrom`, `validTill`, `usageLimit`, `usedCount`, `isActive`, `createdAt`.

15. **Availability**: `_id`, `provider` (ref: Provider), `dayOfWeek` (0-6), `startTime` ("09:00"), `endTime` ("19:00"), `slotDuration` (60), `maxBookingsPerSlot` (default 1), `isOffDay` (Boolean), `customBlockedDates` (array of dates).
16. **ProviderDocument**: `_id`, `provider` (ref: Provider), `documentType` (`['id_proof', 'business_license', 'trade_certificate', 'address_proof']`), `documentUrl`, `documentNumber`, `status` (`['pending', 'approved', 'rejected']`), `rejectionReason`, `verifiedAt`, `createdAt`.
17. **Notification**: `_id`, `recipient` (ref: User), `title`, `message`, `type` (`['booking_update', 'payment_success', 'account_status', 'complaint_update']`), `data` (link or reference ID), `isRead`, `createdAt`.

5. API Endpoint Specifications

Auth Endpoints (`/api/auth`)

- `POST /register`: Registers User + role-specific profile (Customer or Provider).
- `POST /login`: Authenticates credentials, returns user details, access token, sets HTTP-only refresh cookie.
- `POST /refresh`: Issues a fresh access token from refresh token.
- `GET /me`: Returns currently authenticated user context and profile.
- `POST /logout`: Invalidates session/refresh token.

Customer Endpoints (`/api/customers`)

- `GET /profile`, `PUT /profile`: View/update customer profile.
- `GET /addresses`, `POST /addresses`, `PUT /addresses/:id`, `DELETE /addresses/:id`: Manage service addresses.
- `GET /bookings`: Fetch customer's bookings with status filters and pagination.
- `GET /bookings/:id`: Fetch single booking details + live tracking timeline.
- `POST /bookings`: Create atomic booking with slot reservation check.
- `POST /bookings/:id/cancel`: Cancel booking according to cancellation policy.
- `GET /bookings/:id/invoice`: Generate and download structured JSON/HTML invoice data.
- `POST /reviews`: Submit review for completed booking.
- `POST /complaints`: Lodge customer complaint for a booking.

Service Provider Endpoints (`/api/providers`)

- `GET /profile` , `PUT /profile` : Manage business details, service areas, bank info.
- `POST /documents` : Upload KYC verification documents.
- `GET /dashboard-stats` : Metrics (revenue, pending requests, active jobs, rating).
- `GET /services` , `POST /services` , `PUT /services/:id` , `DELETE /services/:id` : Manage offered services.
- `GET /availability` , `PUT /availability` : Configure working hours, slot capacity, blocked days.
- `GET /bookings` : Incoming, active, and completed booking requests.
- `PUT /bookings/:id/accept` : Accept pending booking request.
- `PUT /bookings/:id/reject` : Reject pending booking request with reason.
- `PUT /bookings/:id/assign-technician` : Assign technician to booking.
- `GET /technicians` , `POST /technicians` , `PUT /technicians/:id` , `DELETE /technicians/:id` : Manage provider's technician staff.
- `GET /earnings` : Financial reports, breakdown of completed bookings, platform commission, net payout.
- `GET /reviews` : Customer reviews and reply functionality.

Technician Endpoints (`/api/technicians`)

- `GET /dashboard-stats` : Metrics (assigned jobs today, in-progress, completed).
- `GET /jobs` : List assigned jobs (Pending, Active, Completed).
- `GET /jobs/:id` : View customer location, contact, problem photos, instructions.
- `PUT /jobs/:id/status` : Update job progress (`accepted` -> `on_the_way` -> `arrived` -> `in_progress` -> `completed`).
- `POST /jobs/:id/complete` : Upload proof-of-work images, technician remarks, complete job.

Super Admin Endpoints (`/api/admin`)

- `GET /analytics` : Platform revenue, total bookings, commission earned, role distributions, recent activities.
- `GET /providers` : List all providers with KYC status filter.
- `PUT /providers/:id/verify` : Approve or Reject provider KYC with remarks.
- `GET /customers` : Customer management, search, disable/enable account.
- `GET /technicians` : Global technicians list and status.

- `GET /categories`, `POST /categories`, `PUT /categories/:id`, `DELETE /categories/:id`: Category taxonomy.
- `GET /services`, `PUT /services/:id/toggle`: Platform-wide service oversight.
- `GET /bookings`: Platform-wide booking ledger with filters.
- `GET /payments`, `GET /commissions`: Financial ledger and platform earnings.
- `GET /complaints`, `PUT /complaints/:id/resolve`: Resolve complaints with admin notes.
- `GET /coupons`, `POST /coupons`, `PUT /coupons/:id`, `DELETE /coupons/:id`: Discount campaigns.
- `GET /settings`, `PUT /settings`: Commission rates, platform rules.

Public Endpoints (`/api/public`)

- `GET /categories`: Active categories with service count.
- `GET /services`: Search, filter by category, price, rating, city.
- `GET /services/:id`: Service detail with provider credentials and reviews.
- `GET /providers/:id`: Provider public profile, rating, portfolio.
- `GET /slots/available`: Get real-time available time slots for a provider & date.
- `POST /coupons/validate`: Check coupon code validity and calculate discount.

Payment Endpoints (`/api/payments`)

- `POST /create-order`: Create Razorpay order (or fallback simulation order).
- `POST /verify`: Verify Razorpay signature, mark booking paid, generate commission entry.
- `POST /webhook`: Handle asynchronous payment notifications.

6. Critical Business Rules & Workflows

1. Atomic Booking & Anti-Double-Booking Guard

- When customer selects a Provider, Service, Date, and TimeSlot:
    1. System checks Provider's `Availability` document for the day of week.
    2. System counts existing active bookings (`status` NOT IN `['cancelled', 'rejected']`) for the same `(provider, bookingDate, timeSlot)`.
    3. If count >= `maxBookingsPerSlot`, API rejects with `409 Conflict: "Time slot is fully booked. Please choose another slot."`

4. Insertion uses MongoDB atomic check/transaction or indexed concurrency token to prevent race conditions.

2. State Machine for Booking Lifecycle

```
[Pending]
   |
   ├──> [Rejected] (by Provider)
   |
   └──> [Accepted] (by Provider)
            |
            └──> [Technician Assigned]
                     |
                     └──> [On The Way]
                              |
                              └──> [Arrived]
                                       |
                                       └──> [In Progress]
                                                |
                                                └──> [Completed] (With remarks &
completion photos)
```

- At any stage prior to `On The Way`, Customer can cancel subject to platform cancellation policy.

3. Automated Commission Calculation

- When a booking completes and payment is `paid`:
  - `BasePrice + Tax` = `TotalAmount`
  - `PlatformCommission = TotalAmount * (Provider.commissionRate / 100)`
  - `ProviderPayout = TotalAmount - PlatformCommission`
  - A permanent record is created in `Commission` and `Transaction` collections for clear financial auditing.

7. Phased Implementation Plan

- **Phase 1: Architecture, Scaffolding & Setup**
  - Initialize `backend` with all folders, dependencies (`express`, `mongoose`, `cors`, `helmet`, `dotenv`, `bcryptjs`, `jsonwebtoken`, `express-rate-limit`, `express-validator`, `multer`, `nodemailer`, `razorpay`).

    - Initialize `frontend` with Vite, React, Bootstrap 5, Bootstrap Icons, Axios, React Router DOM, Recharts.
    - Setup unified theme CSS and base styles.
- **Phase 2: Database Models, Relationships & Database Seeder**
    - Implement all 17 Mongoose models with validation, pre-save hooks, methods, and indexes.
    - Create robust, realistic seeder with 4 complete user types (Super Admin, 2 Verified Providers, 3 Technicians, 2 Customers), Categories, Services, Addresses, Bookings across various lifecycle states, Reviews, Coupons, and Complaints.
- **Phase 3: Backend Authentication & Core Services**
    - Auth controller & routes with JWT access/refresh token mechanism.
    - Role-based authorization middleware (`protect` , `authorize('admin')` , etc.).
    - Error handling middleware and standard `apiResponse` helper.
- **Phase 4: Backend Business Modules & REST APIs**
    - Public catalog APIs, Category/Service management.
    - Slot availability engine and atomic booking creation.
    - Provider management (Accept/Reject, Technician Assignment, Earnings).
    - Technician job management (Status progression, proof upload).
    - Customer booking management, cancellations, invoices.
    - Razorpay payment order creation and HMAC-SHA256 signature verification (with auto-test fallback mode).
    - Reviews, Complaints, Coupons, and Super Admin analytics APIs.
- **Phase 5: Frontend Foundations & Shared Architecture**
    - Axios client with token interceptor.
    - `AuthContext` with role state, `ToastContext` with modern popups.
    - Base Layouts (`PublicLayout` , `CustomerLayout` , `ProviderLayout` , `TechnicianLayout` , `AdminLayout`) with responsive sidebar and topbar navigation.
    - Reusable UI widgets: `StatCard` , `DataTable` , `LoadingSpinner` , `EmptyState` , `ConfirmModal` , `StarRating` , `Badge` .
- **Phase 6: Public Portal & Customer Experience**
    - Homepage: Hero with search bar, Category carousel/cards, Featured Services, How it Works, Testimonials, Verified Provider showcase, Footer.

    - Services Catalog with category filters, price filter, search, and sorting.
    - Service Details page with full specs, provider badge, gallery, reviews, and dynamic "Book Now" CTA.
    - Customer Booking Modal / Page with multi-step flow: Address selector/creator -> Date & TimeSlot picker -> Problem description & image upload -> Coupon code application -> Price breakdown -> Payment method selection (Online Razorpay / Pay on Completion).
- **Phase 7: Customer Dashboard**
    - Overview with booking stats and upcoming appointments.
    - My Bookings page with status tabs (All, Active, Completed, Cancelled).
    - Booking Details page with live status stepper, technician details, invoice download modal, review submission modal, and complaint creation.
    - Saved Addresses management and Profile editor.
- **Phase 8: Service Provider Dashboard**
    - KPI cards: Total Revenue, Pending Requests, Active Jobs, Rating.
    - Booking Requests management with 1-click Accept / Reject.
    - Dispatch & Technician assignment modal.
    - Service catalog manager (Add/Edit service, pricing, images).
    - Working Hours & Availability scheduler.
    - Technician team manager (Invite / Add technician, status toggle).
    - Financials & Commission reports with breakdown.
    - Document & KYC verification upload status.
- **Phase 9: Technician Mobile-First Dashboard**
    - Jobs queue (Assigned, Active, Completed).
    - Job execution screen: Customer address with map link, direct phone contact, problem photos.
    - Step-by-step status progression (*On The Way*, *Arrived*, *In Progress*).
    - Job completion form with remarks and completion proof images.
- **Phase 10: Super Admin Control Center**
    - Global Analytics Dashboard with Recharts (monthly platform revenue, booking distribution, top services).
    - Provider Verification Queue (Review business docs, 1-click Approve / Reject with feedback).

    - Platform Taxonomy (Category CRUD, Services catalog oversight).
    - Platform-wide Bookings ledger and Payment / Commission reconciliation.
    - Customer Complaints resolution desk.
    - Promotional Coupons engine (Create, activate/deactivate, usage limits).
- **Phase 11: Verification, Integration Testing & Documentation**
    - Run end-to-end integration tests on all workflows.
    - Verify every user role using the seeded credentials.
    - Provide comprehensive README with architecture, setup instructions, API docs, seed accounts, and deployment guides.